



Assignment_3

Prepared by:

Name of Student : Chaitanya Dalvi

Roll No: 19

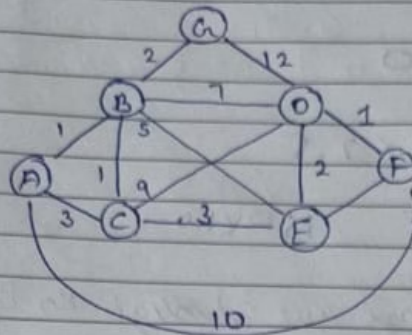
Batch: 2023-27

Dept. of CSE

Assignment 3.

M	T	W	T	F	S	S
Page No.:						YOUVA
Date:						

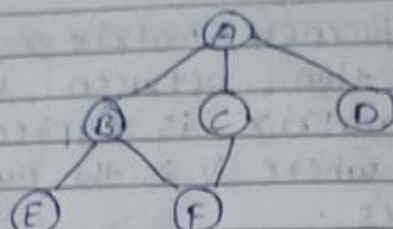
17



Ans

	A	B	C	D	E	F	G
A	0	1	3	1	8	10	10
B	0	1	2	8	6	10	3
C	0	1	2	8	5	10	3
D	0	1	2	8	5	10	3
E	0	1	2	7	5	7	3
F	0	1	2	7	5	7	3
G	0	1	2	7	5	7	3

2]



=> Step 1:- Go one after another depth till the node are unvisited & it is deadend

Step 2:- Add one by one on Stack

Step 3:- Once deadend meet backtrack

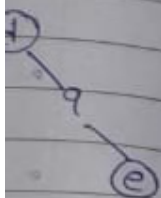
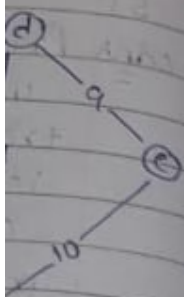
Step 4:- then check their nodes if unvisited

Step 5:- For backtracking, pop from Stack

Step 6:- if met then add in stack

Step 7:- after popping out all the ~~tracks~~ nodes from Stack we will get DFS

eg:- ABFCED



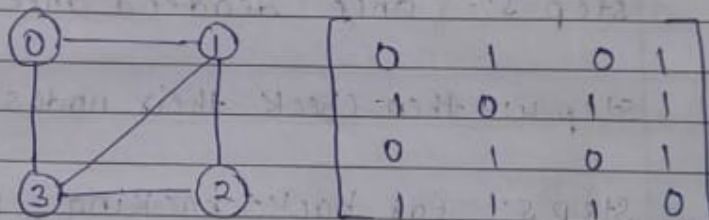
3]

Ans i] In adjacency matrix represent the connection between vertices (nodes) the matrix is typically of size $V \times V$, where V is the number of vertices.

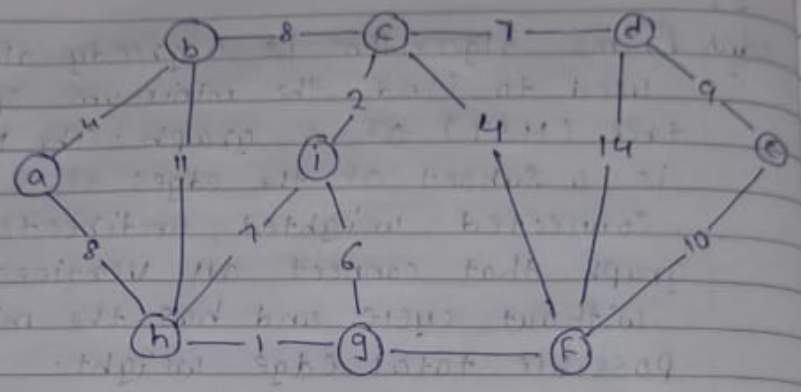
ii] For an undirected graph, the matrix will be Symmetric.

iii] For a directed graph, the adjacency matrix is not necessary Symmetric.

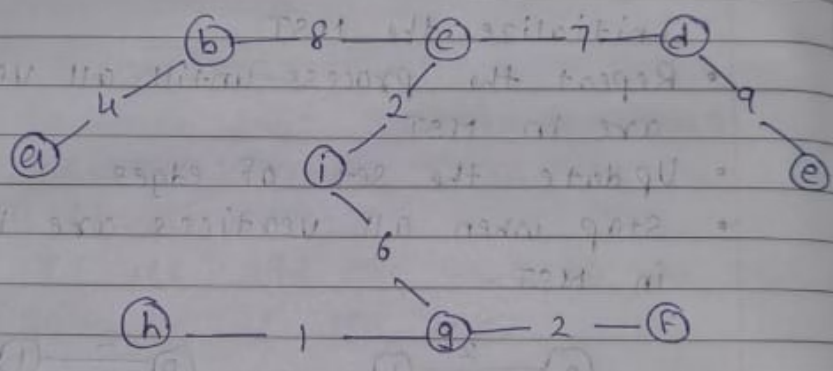
eg:-



4] Ans:



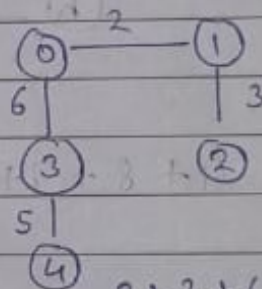
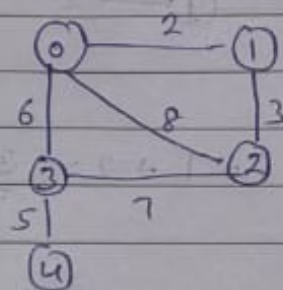
Ans:



$$4 + 8 + 7 + 9 + 2 + 6 + 1 + 2 = 39$$

5] Ans Prim's algorithm is a greedy algorithm used to find the minimum spanning tree (MST) of a graph. The MST is a subset of the edges of a connected, weighted, undirected graph that connect all vertices without cycle and has the minimum possible total edge weight.

- Initialize the MST
- Repeat the process until all vertices are in MST
- Update the set of edges
- Stop when all vertices are included in MST.



$$2 + 3 + 6 + 5 = 16$$

6] Ans

Kruskal's algorithm

1] sort Edges:

- sort all edges by edges by ascending weight

2] Initialize MST

- Start with an empty MST. Treat each vertex as its own disjoint set

3] Process Edge

For each edge, check if the two connected vertices belong to the different sets:

- If yes, add the edge to the MST and merge the sets
- If no, discard the edges to avoid cycle

7]

Ques

The Bellman Ford algorithm finds the shortest path from a single source vertex to all other vertices in a weighted graph, even with negative weights.

Steps:

1] Initialize distances:-

Set the source vertex distance 0 and all others to infinity.

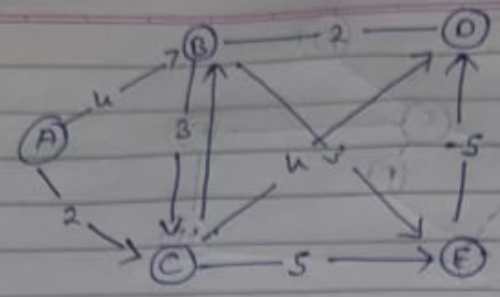
2] Relax edges:-

For each edge, update the distance to the destination vertex if the current path offers a shorter route.

3] Check for the negative cycle

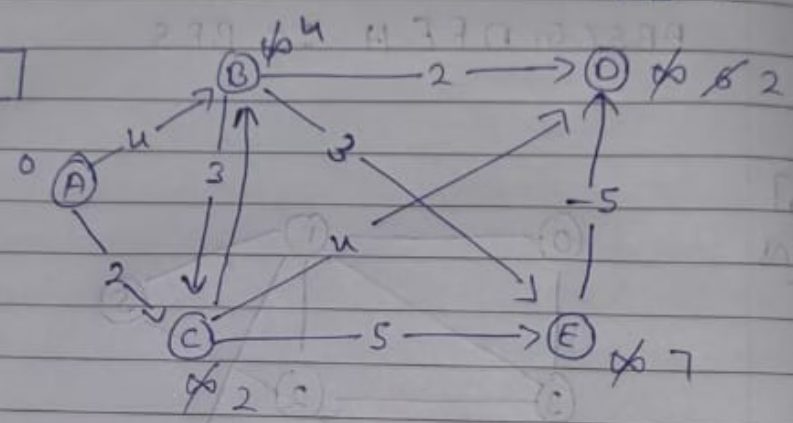
After the $V-1$ relaxation, check if any edge can still be relaxed.

8]
 Wds

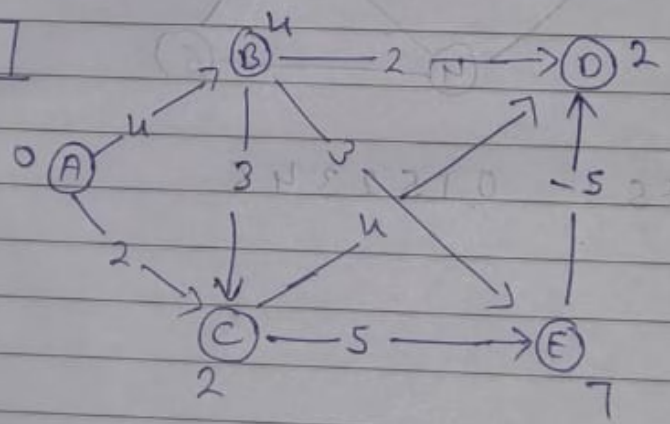


- (AB,
- AC,
- BC,
- BD,
- BE
- CB
- CE
- CD
- ED)

R1



R2



A 0	C 2	E 7
B 4	D 2	

